

Лабораторная работа 8

ПРОСТЫЕ ПРОГРАММЫ НА АССЕМБЛЕРЕ

Напишите на ассемблере и поместите в файл Video_io.asm программу вывода двузначного шестнадцатеричного числа. Все процедуры исходной программы должны содержать вводные и текущие комментарии.

Получите файл Video_io.com и, используя Debug, тщательно протестируйте программу, меняя 3Fh по адресу 101h на каждое из граничных условий, которые использовались ранее в п.6 для проверки программы, выполняющей те же функции. Обязательно сохраните файл Video_io.asm.

На последующих работах мы будем добавлять в него новые процедуры. Пользуясь этими процедурами как "кирпичиками", мы сможем создавать достаточно сложные программы. Примечание. Для отладки Вашей программы удобно использовать подход, называемый "снизу-вверх".

Согласно ему сначала отлаживаются процедуры, расположенные внизу дерева подпрограмм. После того, как эти процедуры отлажены, отлаживаются процедуры их вызывающие и так далее. Реализация данного подхода предполагает первоначальную корректировку процедуры Test_write_byte_hex так, чтобы из нее вызывалась не процедура Write_byte_hex, а процедура Write_digit_hex. После того как Write_digit_hex будет отлажена, Test_write_byte_hex восстанавливается, и программа отлаживается целиком.

Ход работы:

```
section .data
    hex_digit db '00$', 0

section .text
global _start

_start:
    mov eax, 4          ; Функция вывода на экран (write)
    mov ebx, 1          ; Файловый дескриптор STDOUT (стандартный вывод)
    mov ecx, hex_digit  ; Указатель на строку для вывода
    mov edx, 3          ; Количество байт для вывода (два символа числа + завершающий нулевой символ)
    int 0x80            ; Вызов системного вызова

    mov eax, 1          ; Функция выхода из программы (exit)
    xor ebx, ebx        ; Код возврата 0
    int 0x80            ; Вызов системного вызова
```

Результат работы программы:

Compilation time: 0.14 sec, absolute running time: 0.16 sec, cpu time: 0.01 sec, memory peak: 6 Mb, absolute service time: 0,5 sec

00\$